Control Flow - Cond

Created On: Oct 03, 2023 | Last Updated On: Feb 14, 2026

`torch.cond` is a structured control flow operator. It can be used to specify if-else like control flow and can logically be seen as implemented as follows.

```
def cond(
    pred: Union[bool, torch.Tensor],
    true_fn: Callable,
    false_fn: Callable,
    operands: Tuple[torch.Tensor]
):
    if pred:
        return true_fn(*operands)
    else:
        return false_fn(*operands)
```

Its unique power lies in its ability of expressing **data-dependent control flow**: it lowers to a conditional operator (`torch.ops.higher_order.cond`), which preserves predicate, true function and false functions. This unlocks great flexibility in writing and deploying models that change model architecture based on the **value** or **shape** of inputs or intermediate outputs of tensor operations.

Warning

`torch.cond` is a prototype feature in PyTorch. It has limited support for input and output types. Please look forward to a more stable implementation in a future version of PyTorch. Read more about feature classification at: https://pytorch.org/blog/pytorch-feature-classification-changes/#prototype

Examples

Below is an example that uses cond to branch based on input shape:

Ask AI

```
import torch

def true_fn(x: torch.Tensor):
    return x.cos()

def false_fn(x: torch.Tensor):
    return x.sin()

class DynamicShapeCondPredicate(torch.nn.Module):
    """
    A basic usage of cond based on dynamic shape predicate.
    """

    def __init__(self):
        super().__init__()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.cond(x.shape[0] > 4, true_fn, false_fn, (x,))

dyn_shape_mod = DynamicShapeCondPredicate()
```

We can eagerly run the model and expect the results vary based on input shape:

```
inp = torch.randn(3)
inp2 = torch.randn(5)
print(dyn_shape_mod(inp), false_fn(inp))
print(dyn_shape_mod(inp2), true_fn(inp2))
```

We can export the model for further transformations and deployment. This gives us an exported program as shown below:

```
inp = torch.randn(4, 3)
ep = torch.export.export(
    DynamicShapeCondPredicate(),
    (inp,),
    dynamic_shapes={"x": {0: torch.export.Dim.DYNAMIC}}
)
print(ep)
```

Notice that `torch.cond` is lowered to `torch.ops.higher_order.cond`, its predicate becomes a Symbolic expression over the shape of input, and branch functions becomes two sub-graph attributes of the top level graph module.

Ask AI

Here is another example that showcases how to express a data-dependent control flow:

```
def true_fn(x: torch.Tensor):
    return x.cos() + x.sin()

def false_fn(x: torch.Tensor):
    return x.sin()

class DataDependentCondPredicate(torch.nn.Module):
    """
    A basic usage of cond based on data dependent predicate.
    """
    def __init__(self):
        super().__init__()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.cond(x.sum() > 4.0, true_fn, false_fn, (x,))

inp = torch.randn(4, 3)
ep = torch.export.export(DataDependentCondPredicate(), (inp,), dynamic_shapes
print(ep)
```

Invariants of torch.ops.higher_order.cond

There are several useful invariants for `torch.ops.higher_order.cond`:

- For predicate:
  - Dynamicness of predicate is preserved (e.g. `gt` shown in the above example)
  - If the predicate in user-program is constant (e.g. a python bool constant), the `pred` of the operator will be a constant.
- For branches:
  - The input and output signature will be a flattened tuple.
  - They are `torch.fx.GraphModule`.
  - Closures in original function becomes explicit inputs. No closures.
  - No mutations on inputs or globals are allowed.
- For operands:
  - It will also be a flat tuple.
- Nesting of `torch.cond` in user program becomes nested graph mo

Ask AI

API Reference

torch._higher_order_ops.cond.cond(*pred*, *true_fn*, *false_fn*, *operands=()*) [source]

Conditionally applies *true_fn* or *false_fn*.

> **Warning**
>
> *torch.cond* is a prototype feature in PyTorch. It has limited support for input and output types. Please look forward to a more stable implementation in a future version of PyTorch. Read more about feature classification at: https://pytorch.org/blog/pytorch-feature-classification-changes/#prototype

cond is structured control flow operator. That is, it is like a Python if-statement, but has restrictions on *true_fn*, *false_fn*, and *operands* that enable it to be capturable using torch.compile and torch.export.

Assuming the constraints on *cond*'s arguments are met, *cond* is equivalent to the following:

```
def cond(pred, true_branch, false_branch, operands):
    if pred:
        return true_branch(*operands)
    else:
        return false_branch(*operands)
```

Parameters:

- **pred** (*Union[bool, torch.Tensor]*) – A boolean expression or a tensor with one element, indicating which branch function to apply.
- **true_fn** (*Callable*) – A callable function (a -> b) that is within the scope that is being traced.
- **false_fn** (*Callable*) – A callable function (a -> b) that is within the scope that is being traced. The true branch and false branch must have consistent input and outputs, meaning the inputs have to be the same, and the outputs have to be the same type and shape. Int output is also allowed. We'll make the output dynamic by turning it into a symint.
- **operands** (*Tuple of possibly nested dict/list/tuple of torch.* of inputs to the true/false functions. It can be empty if true_fn/false_fn doesn't require input. Defaults to ().

Ask AI

Return type:

Any

Example:

```
def true_fn(x: torch.Tensor):
    return x.cos()


def false_fn(x: torch.Tensor):
    return x.sin()


return cond(x.shape[0] > 4, true_fn, false_fn, (x,))
```

Restrictions:

- The conditional statement (aka *pred*) must meet one of the following constraints:
  - It's a *torch.Tensor* with only one element, and torch.bool dtype
  - It's a boolean expression, e.g. *x.shape[0] > 10* or *x.dim() > 1 and x.shape[1] > 10*
- The branch function (aka *true_fn/false_fn*) must meet all of the following constraints:
  - The function signature must match with operands.
  - The function must return a tensor with the same metadata, e.g. shape, dtype, etc.
  - The function cannot have in-place mutations on global variables. (Note: in-place tensor operations such as *add_* for intermediate results are allowed in a branch)
  - The function can perform in-place mutations on its input tensors during inference (i.e., when *torch.is_grad_enabled()* is False). Note: When using *torch.compile()* with a non-constant predicate, the outputs will always be new tensors that do not share object identity with the original inputs.

    Example:

Ask AI

```
def true_fn(x):
    return x.sin_()


def false_fn(x):
    return x + 1


def f(x):
    return cond(x.sum() > 0, true_fn, false_fn, (x,))


x = torch.ones(4)
with torch.no_grad():
    result = torch.compile(f)(x)
assert result is not x  # result is a new tensor, not the ori
```


Docs

Access comprehensive developer documentation for PyTorch

View Docs

Tutorials

Get in-depth tutorials for beginners and advanced developers

View Tutorials

Resources

Find development resources and get your questions answered

View Resources

Stay in touch for updates, event info, and the latest news

First Name* Last Name* Email*

Select Country* SUBMIT

Ask AI

By submitting this form, I consent to receive marketing emails from the LF and its projects regarding their events, training, research, developments, and related announcements. I understand

that I can unsubscribe at any time using the links in the footers of the emails I receive. Privacy Policy.



© PyTorch. Copyright © The Linux Foundation®. All rights reserved. The Linux Foundation has registered trademarks and uses trademarks. For more information, including terms of use, privacy policy, and trademark usage, please see our Policies page. Trademark Usage. Privacy Policy.

Ask AI